Home-Lab Umstellung: Schritt-für-Schritt-Anleitung mit Befehlen

■ **Dokument-Version: v1.2.3** · Stand: 2026-11-01 · Dieses Dokument ist die **aktuellste Version**.

Änderungshistorie (Changelog)

| Version | Datum | Änderungen |

|---|---|---|

| **v1.2.3 (aktuell)** | 2026-11-01 | Migrations-Sonntag-Ergebnisse: Phase-0-Abnahme erteilt (SMART-Gate sdb via Zonentest-Kompensation 279GiB @ 84,7 MB/s fehlerfrei); restic-Initial 536,6 GiB auf Hetzner erfolgreich, Cron scharf; verbindliches Storage-Layout (Dataset-Tabelle mit Besitzern: photos/documents/cloud/media/cameras/backup); tank/cloud-Definition (Sync-Dateien, kein Archiv); Dokumenten-Struktur ulli/nadine/familie; Paperless-Philosophie A (DB-zentriert, Storage-Paths, Altdaten-Import, DB-Dump im Backup); Vaultwarden-first-Prinzip + Credential-Handling Migrationsphase; Hardware: GPU + Coral ausgebaut (Frigate-Relikte), 10G-Netzwerkkarte verworfen (Engpassanalyse), Investitionspriorität RAM/Storage; Unraid-Cron-Syntax-Warnung (kein Benutzerfeld!); SSH-Port 2002 + Key-Auth; Dashboard status-cli.sh; logrotate-Baustein für Phase 3 |

| v1.2.2 | 2026-10-31 |

| **v1.2.2 (aktuell)** | 2026-10-31 | Praxis-Erkenntnisse eingearbeitet: Inventar-Tabelle 0.0a-1 mit erfassten Seriennummern/by-IDs (WD30EFAX x2, CT1000P3SSD8, MQ01ABD100); Hetzner-Sub-Account-SSH-Key-Prozedur (Key via SFTP in ~/.ssh/authorized_keys, Sub-Account-Domain u706080-sub1.your-storagebox.de); Unraid-Hinweis /tmp=RAM (keine großen Restore-Ziele); domains/Macinabox wird nicht migriert (nur lokale Archiv-Kopie); Restore-Repeat mit Macinabox-Exclude |

| v1.2.1 | 2026-10-31 | Gemeinsames Dokument: PDF aus Markdown erzeugt, altes Roadmap-PDF eingestellt |

| **v1.2.0 (aktuell)** | 2026-10-30 | Beschlüsse aus Vergleich mit Alternativ-Anleitung eingearbeitet: Alt-Platten bleiben EINGEBAUT (2.1a umgeschrieben, Migration über temporären bulk-Pool); tank-Mirror-Endziel + dedizierter backup-Pool mit strikter Abfolge (3.1); sanoid/syncoid-Nächtliche Replikation inkl. readonly-Schutz (3.1a); externe USB-Platte als dauerhaftes lokales restic-Tier (3.1b); DMZ-Option B: Futro in eigenem VLAN mit 5-Regel-Set (0.5 erweitert); TrueNAS als Option geprüft und bewusst verworfen (Architektur-Notiz) |

| v1.1.0 | 2026-10-30 | Migrationssicherheit ergänzt: 0.0a Inventur, 0.0b SMART-Gate, 0.0c Container-Doku, 0.0d Flash-Backup, 0.0e Secrets-Export; 2.1 rsync-Verifikation + externe Backup-Platte, 2.1a Alt-Platten-Rollback, 2.9a 14-Tage-Beobachtung; 3.4 Platten-Freigabe, 3.5 nmap-Check; Platzhalter-Tabelle erweitert |

| v1.0.0 | 2026-10-30 | Initiale Version: 4 Phasen mit Befehlen (Hetzner/restic, Futro-Edge-Box, Proxmox-Migration, Finalisierung), Abschlusstests je Phase |

Git-Tags: Der Stand jeder Version ist über ein Git-Tag (`v1.0.0`, `v1.1.0`, …) im Repository markiert. Änderungen zwischen Versionen: `git diff v1.0.0 v1.1.0` bzw. auf GitHub unter *Releases/Tags* bzw. im Commit-Vergleich.

Platzhalter-Konvention: Alle Werte in <. . .> sind durch deine individuellen Werte zu ersetzen:

> | Platzhalter | Bedeutung | Beispiel |
> |---|---|---|
> | `<DOMAIN>` | Deine externe Domain | `example.duckdns.org` |
> | `<FUTRO_IP>` | Feste IP des Futro | `192.168.10.5` |
> | `<SRV_IP>` | Feste IP des Proxmox-Hauptserver | `192.168.10.10` |
> | `<T630_IP>` | Feste IP des T630 | `192.168.10.20` |
> | `<USER>` | Dein sudo-Benutzer | `admin` |
> | `<WG_PORT>` | WireGuard-Port | `51820` |
> | `<HETZNER_USER>` | Hetzner Storage Box Benutzer | `u12345` |
> | `<HBX_HOST>` | Hetzner Storage Box Host | `u12345.your-storagebox.de` |
> | `<HETZNER_SUBID>` | Storage Box Sub-ID | `s12345` |
> | `<NETZ>` | Dein LAN-Subnetz | `192.168.10.0/24` |
> | `<WG_NETZ>` | WireGuard-Subnetz | `10.10.0.0/24` |
> | `<ARRAY_HDD1_ID>` / `<ARRAY_HDD2_ID>` | by-id-Kennungen der Array-Platten – **erfasst**: `ata-WDC_WD30EFAX-68JH4N0_WD-WX12D4071P2X` / `ata-WDC_WD30EFAX-68JH4N1_WD-WX92D91FJ49T` | siehe Inventar 0.0a-1 |
> | `<TEMP_HDD_ID>` | by-id-Kennung der bisher ungenutzten 3-TB-HDD (temp. bulk) | `ata-WDC_WD30..._YYYY` |
> | `<USB_DISK>` / `<USB_UUID>` | Externe USB-Backup-Platte / ihre UUID (`blkid`) | `/dev/sde` |
> | `<UNRAID_IP>` | IP des Unraid-Servers (heute) | `192.168.10.30` |
> | `<EXT_DEV>` | Externe Backup-Platte | `/dev/sdE` |
> | `<HDD_DEV>` | 3-TB-HDD (Pool-Kandidat) | `/dev/sdb` |
> | `<GATEWAY_IP>` | Gateway (UDM Pro) | `192.168.10.1` |
> | `<SHARE_MIT_PLATZ>` | Unraid-Share mit Platz für Doku/Backups | `backup` |

Golden Rule vor jedem Schritt: Aktuelles Backup vorhanden? Dann erst weitermachen.

Verbindliche Architektur-Beschlüsse (Phase-0-Abnahme 2026-11-01)

A. Storage-Layout tank (verbindlich für Phase 2.3/3.1)

Dataset	Besitzer (schreibend)	Konsumenten (lesend)	Backup-Priorität
`tank/photos`	Immich	HA, Nextcloud via External Storage	hoch (production-Template)
`tank/documents`	Paperless	Nextcloud via External Storage	hoch
`tank/cloud`	Nextcloud	–	hoch
`tank/media/movies`	– (manuelle Ablage)	Jellyfin (perspektivisch)	mittel
`tank/media/music`	– (manuelle Ablage)	Music Assistant, Jellyfin	mittel
`tank/cameras`	Reolink (SMB)	HA, 14-Tage-Cleanup	niedrig (kein Offsite)
`tank/backup`	Proxmox-Host	–	Systemsicherung

Regel: Pro Dataset genau EIN schreibender Besitzer – alle anderen Zugriffe read-only oder via External-Storage-Bindung.

`tank/cloud`**-Definition:** Ausschließlich Nextcloud-Sync-Dateien (persönliche Dokumente, Sync-Ordner, kleine App-Syncs). KEIN Archiv, KEINE Fotos (→ photos), KEINE Medien (→ media), KEINE Dienst-Datenbanken.

Dokumenten-Struktur (wird von Paperless-Storage-Paths generiert, nicht manuell gepflegt):

```
tank/documents/
■■■ ulli/        (Tag-basiert generiert)
■■■ nadine/
■■■ familie/     (wohnen, versicherungen, fahrzeuge, gesundheit...)
■■■ _archiv/
```

B. Paperless-Philosophie A: datenbankzentriert (Beschluss)

• Paperless verwaltet ALLE Dokumente (WebUI/App als primärer Zugang)

• Struktur entsteht über Tags (ulli/nadine/familie) + Dokumenttypen + **Storage-Paths**:

`tank/documents/{tag_person}/{document_type}/{created_year}/{title}`

• Ordner auf tank/documents sind das automatisch generierte Abbild, NICHT manuell sortieren

• Altdaten-Migration (120 GB): Import über Paperless-Mechanik (consume), NICHT direkt in originals/ kopieren

• Backup-Skript-Erweiterung: `pg_dump` der Paperless-DB vor dem restic-Datei-Backup

• Nextcloud-Anbindung: tank/documents als External Storage (lesen + Scan-Ablage in Consume-Bereich)

C. Migrationsreihenfolge: Vaultwarden-first (Beschluss)

1. Proxmox-Host installiert + gehärtet (2FA, SSH-Key)

2. ZFS-Pools + Datasets

3. WireGuard-Zugang (UDM) verifiziert

4. **Vaultwarden-LXC – Prio 1** (Restore aus restic-Bestand, VPN-only)

5. NPM + CrowdSec auf Futro (Phase 1)

6. Weitere LXCs (Paperless, Nextcloud, Immich, Traccar) – Credentials direkt in Vaultwarden

Credential-Handling Migrationsphase: Übergangs-Credentials verschlüsselt auf Laptop (KeePassXC/gpg-Zip, NIEMALS Klartext), nach Vaultwarden-Livegabe einpflegen, Übergangsdatei löschen. Ausgenommen von Vaultwarden: restic-Passphrase (bleibt auf den 2 USB-Sticks, Henne-Ei-Prinzip).

D. Hardware-Beschlüsse (2026-11-01)

Beschluss	Begründung
GPU ausbauen (einmotten, antistatisch)	Frigate gestrichen – kein Zweck; Reserve für späteres evtl. Jellyfin-Transcoding
Coral ausbauen (einmotten)	Frigate-Relikt, kein Zweck im Zielbild
10G-Netzwerkkarte verworfen	Engpassanalyse: 2.5GbE > alle realen Workloads (WAN 300M, Clients GbE, HDDs ~180 MB/s)
Investitionspriorität: RAM (2×32 GB erst bei ARC-Trefferquote <90 %) → Storage	Realistische Workload-Analyse

Beschluss	Begründung
Schrank-Fund: 3-TB-HDD (WD Red/IronWolf)	Pool-Kandidat #3, beim Aufschrauben anschließen + SMART-testen

Aufschraub-Checkliste: Schrank-3TB + dritte HDD + 250-GB-SSD einbauen · GPU + Coral ausbauen · Seriennummern/by-IDs erfassen · SMART-Tests der Neuzugänge

E. Betriebliche Lektionen aus Phase 0 (Unraid-spezifisch)

1. **Unraid-cron.d-Dateien haben KEIN Benutzerfeld!** Debian-Syntax … `root befehl` erzeugt Minutentakt-Spam (`root: command not found`). Syntax: `0 3 * * * bash /pfad/skript.sh`

2. **Unraid-/tmp liegt im RAM** – keine großen Restore-Ziele dorthin (Sparse-Dateien sprengen das RAM)

3. SSH aktiv mit Port 2002 + Key-Auth (Laptop-Key hinterlegt)

4. Status-Dashboard: `bash /boot/custom/homelab-status/status-cli.sh` – Aufruf vom Laptop via `watch -n 30 'ssh -p 2002 root@<UNRAID_IP> bash /boot/custom/homelab-status/status-cli.sh'`

5. Hetzner-Storage-Box-Sub-Accounts: eigene Domain (`uXXXXX-sub1.your-storagebox.de`), Key via SFTP in `~/.ssh/authorized_keys`, restricted Shell (keine Pipes/Redirects)

6. SMART-Longtest-WD30EFAX-Quirk: Hänger bei 10 % Remaining ohne Plattendefekt – Kompensation via dd-Zonen-Vollltest (279 GiB @ 84,7 MB/s fehlerfrei = Abnahme mit Vermerk)

7. Dashboard/Webserver-Versuche auf Unraid obsolet (kein busybox/python im PATH) – CLI-Dashboard via SSH ist der Weg

F. Logrotate-Baustein (Phase 3, auf Proxmox-Host)

```
# /etc/logrotate.d/restic-backup
/var/log/restic.log /var/log/restic-first.log {
    weekly
    rotate 8
    missingok
    notifempty
    compress
    delaycompress
}
```

Leitlinie: Datensicherheit vor Geschwindigkeit.

Rollback-Prinzip (wichtigstes Sicherheitsnetz der gesamten Migration): Die alten

Unraid-Datenplatten werden beim Umbau **ausgebaut, beschriftet und antistatisch gelagert**

und bleiben **UNANGETASTET**, bis alle Abschlusstests von Phase 3 bestanden sind.

Notfall-Rollback = Platten wieder einbauen, Unraid-Stick booten, Array starten – fertig.

Erst danach werden sie gelöscht oder einem Pool zugeführt (siehe 3.4).

Phase 0 – Vorbereitung, Backup-Basis & UDM-Hardening

0.0a Bestandsaufnahme & Inventur (auf dem Unraid-Server)

```
ssh root@<UNRAID_IP>
mkdir -p /boot/migration-doku

# Platten, Geräte, Seriennummern erfassen und dokumentieren:
lsblk -o NAME,SIZE,MODEL,SERIAL,MOUNTPOINT | tee /boot/migration-doku/lsblk.txt

df -h /mnt/user /mnt/cache

# Datenmengen je Share (für Zeitplanung des rsync):
du -sh /mnt/user/* | sort -h | tee /boot/migration-doku/du-shares.txt
```

0.0a-1 Inventar-Tabelle (erfasst am 2026-10-31, Host "Smartie")

Gerät	Modell	Seriennummer	by-id (für zpool create)	Rolle (Ziel)
sdb	WDC WD30EFAX-68JH4N0 (2,7 TB)	WD-WX12D4071P2X	`ata-WDC_WD30EFAX-68JH4N0_WD-WX12D4071P2X`	Unraid-Array → später **tank-Mirror** (1/2)
sdc	WDC WD30EFAX-68JH4N1 (2,7 TB)	WD-WX92D91FJ49T	`ata-WDC_WD30EFAX-68JH4N1_WD-WX92D91FJ49T`	Unraid-Array → später **tank-Mirror** (2/2)
– (nicht angeschlossen)	WDC 3-TB-HDD	**noch zu erfassen**	nach Anschluss via `ls -l /dev/disk/by-id/`	temporärer **bulk**-Pool → später **backup**-Pool
– (nicht angeschlossen)	250-GB-SSD	**noch zu erfassen**	nach Anschluss erfassen	**Proxmox-OS-Boot** (Phase 2.2)
nvme0n1	Crucial CT1000P3SSD8 (1 TB)	2320E6D694BB	`nvme-CT1000P3SSD8_2320E6D694BB`	**fast**-Pool (VM/LXC-Disks, DBs)
sdf	Toshiba MQ01ABD100 (931,5 GB, USB-Intenso-Gehäuse)	X5VBW9D3T	`wwn-0x5000039694984922`	**Externe Backup-Platte** (lokale 2. Kopie, rsync-Ziel)
sda	Intenso Micro Line (3,7 GB)	04EC7300000007E9	`usb-Intenso_Micro_Line_04EC7300000007E9-0:0`	**Unraid-Boot-Stick** (im Server; archivieren nach Migration)

> **Hinweise:** (1) `sdX`-Namen können sich nach Reboots ändern – für `zpool create` ausschließlich die by-id-Namen verwenden. (2) Die Platzhalter `<ARRAY_HDD1_ID>`/`<ARRAY_HDD2_ID>` in Phase 3.1 sind damit aufgelöst: `ata-WDC_WD30EFAX-68JH4N0_WD-WX12D4071P2X` und `ata-WDC_WD30EFAX-68JH4N1_WD-WX92D91FJ49T`. (3) Nach dem Anschluss der dritten HDD und der 250-GB-SSD: Seriennummern hier nachtragen (Befehl: `ls -l /dev/disk/by-id/ | grep -v part`). (4) Der Passphrase-Stick "restic-key2" (Micro Line 3,7 GB – Seriennummer beim nächsten Einstecken erfassen) trägt die Restic-Passphrase; Verwahrung getrennt von "restic-key1" (14,7 GB Silicon-Power).

0.0b SMART-Gate (nur einwandfreie Platten wandern in den ZFS-Pool)

**Kriterium: Reallocated / Pending / Uncorrectable = 0. Platten mit Wert > 0 NICHT

für den Pool verwenden.**

```
apt install -y smartmontools   # falls nicht vorhanden

for d in /dev/sd? /dev/nvme?n1; do
  echo "=== $d ==="
  smartctl -H -A $d | grep -E "overall|Reallocated|Pending|Uncorrectable|Power_On|Percentage Used|Media and Data"
done | tee /boot/migration-doku/smart-uebersicht.txt

# Für die 3 zukünftigen Pool-HDDs: Langen Selbsttest ausführen (dauert Stunden!)
smartctl -t long /dev/<HDD_DEV>
# später Ergebnis prüfen:
smartctl -l selftest /dev/<HDD_DEV>
```

0.0c Docker-Container dokumentieren (Vorbereitung der Migration)

```
docker ps -a --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}" \
  > /boot/migration-doku/container-liste.txt

for c in $(docker ps -aq); do
  docker inspect $c > /boot/migration-doku/inspect-$(docker inspect --format '{{.Name}}' $c | tr -d '/').json
done

# Unraid-Docker-Templates mitkopieren (enthält alle ENV/Volumes/Ports!):
cp -r /boot/config/plugins/dockerMan/templates-user /boot/migration-doku/templates

# Doku AUSSERHALB des Servers sichern (Laptop o. ä.):
# scp -r root@<UNRAID_IP>:/boot/migration-doku ~/migration-doku/
```

0.0d Unraid-Flash-Backup

```
cp -r /boot /mnt/user/<SHARE_MIT_PLATZ>/unraid-flash-backup-$(date +%F)
# ZUSÄTZLICH über die GUI: Main > Flash > Flash Backup
# Datei außerhalb von Unraid ablegen (Laptop, externe Platte)
```

0.0e Geheimnisse/Secrets sichern (vor der Migration, PFLICHT!)

Diese Werte gehen bei einer Neu-Installation verloren und müssen extern gesichert sein:

Was	Wie sichern
Vaultwarden Vault	GUI: Tools > Export Vault > **.json Encrypted** (Passwort-Wortfolge separat notieren)
Vaultwarden `ADMIN_TOKEN`, `rsa_key.*`	aus Appdata-Verzeichnis, in Passwortmanager
Nextcloud `config.php`	`passwordsalt`, `secret`, `instanceid` (in Passwortmanager, nie im Klartext in Git)
Paperless `PAPERLESS_SECRET_KEY`	aus Docker-Template/ENV, in Passwortmanager
DDNS-Updater API-Token	Provider-Konsole bzw. ENV
CrowdSec Enrollment-Key	`cscli`-Ausgabe bzw. Config
Restic-Passphrase	USB-Stick (0.2) + Passwortmanager als Backup

```
# Beispiel Vaultwarden-Keys aus dem Unraid-Appdata-Verzeichnis:
tar czf /mnt/user/<SHARE_MIT_PLATZ>/vaultwarden-keys-$(date +%F).tgz \
  /mnt/user/appdata/vaultwarden/
# Achtung: diese Datei NICHT unverschlüsselt in Git oder Cloud ablegen!
# Auf dem Laptop verschlüsselt ablegen (z. B. 7z/gpg).
```

0.1 Hetzner Storage Box & restic einrichten (auf dem aktuellen Unraid-Server bzw. späteren Hauptserver)

```
# restic installieren (Debian/Ubuntu)
sudo apt update && sudo apt install -y restic

# SSH-Zugang zur Storage Box testen (Hetzner: ssh auf Port 23, Subuser anlegen!)
# Wichtig: In der Hetzner-Konsole einen Sub-Account mit Subpath anlegen,
# z. B. /home/<HETZNER_SUBID>/restic  und SSH-Key-Auth aktivieren
ssh -p 23 <HETZNER_USER>@<HBX_HOST> "mkdir -p restic"

# SSH-Key fuer die Box erzeugen (nur fuer Backup, ohne Passphrase, da Cron-Job)
ssh-keygen -t ed25519 -f ~/.ssh/hetzner_bfx -N ""
ssh-copy-id -p 23 -i ~/.ssh/hetzner_bfx.pub <HETZNER_USER>@<HBX_HOST>

# ~/.ssh/config Eintrag ergaenzen:
cat >> ~/.ssh/config <<'EOF'
Host hbx
    HostName <HBX_HOST>
    User <HETZNER_USER>
    Port 23
    IdentityFile ~/.ssh/hetzner_bfx
EOF
chmod 600 ~/.ssh/config

# Restic-Repository initialisieren (verschluesselt!)
# PASSPHRASE: lang & zufaellig erzeugen und auf USB-Stick sichern (siehe 0.2)
openssl rand -base64 32   # -> Ausgabe = <RESTIC_PASSPHRASE>
export RESTIC_REPOSITORY="sftp:hbx:restic"
export RESTIC_PASSWORD="<RESTIC_PASSPHRASE>"
restic init
```

0.2 Passphrase/Key auf USB-Stick verwahren

```
# Zwei USB-Sticks mounten (z. B. /media/usb1 und /media/usb2), dann:
echo "<RESTIC_PASSPHRASE>" > /media/usb1/restic_key.txt
echo "<RESTIC_PASSPHRASE>" > /media/usb2/restic_key.txt
chmod 600 /media/usb1/restic_key.txt /media/usb2/restic_key.txt
# Stick 1: Tresor o. ae. - Stick 2: anderer Ort. NICHT auf Server lassen, NICHT in Vaultwarden.
```

0.3 Backup-Job einrichten (Cron)

```
sudo tee /etc/restic-env <<'EOF'
export RESTIC_REPOSITORY="sftp:hbx:restic"
export RESTIC_PASSWORD="<RESTIC_PASSPHRASE>"
EOF
sudo chmod 600 /etc/restic-env

# Backup-Skript
sudo tee /usr/local/bin/backup-offsite.sh <<'EOF'
#!/bin/bash
set -e
source /etc/restic-env
# Pfade anpassen: Unraid-Shares bzw. spaeter Proxmox-Datasets
restic backup /mnt/user/nextcloud /mnt/user/paperless /mnt/user/vaultwarden \
  --tag daily
# Alte Snapshots aufraeumen (Keep: 7 Tage, 4 Wochen, 6 Monate)
restic forget --keep-daily 7 --keep-weekly 4 --keep-monthly 6 --prune
EOF
sudo chmod +x /usr/local/bin/backup-offsite.sh

# Cron: taeglich 02:00 Uhr
echo "0 2 * * * root /usr/local/bin/backup-offsite.sh >> /var/log/restic.log 2>&1" \
  | sudo tee /etc/cron.d/restic-backup
```

0.4 Restore-Test (PFLICHT vor Weitermachen!)

```
source /etc/restic-env
restic snapshots                      # Backups listen
restic restore latest --target /tmp/restore-test --path /mnt/user/vaultwarden
diff -r /mnt/user/vaultwarden /tmp/restore-test/mnt/user/vaultwarden && echo "RESTORE OK"
rm -rf /tmp/restore-test
```

0.5 Netzwerk-/IP-Plan dokumentieren – inkl. DMZ für den Futro (Beschluss v1.2.0, Option B)

```
Trusted VLAN 10: 192.168.10.0/24   (Laptops, Handys, Hauptserver, T630)
DMZ VLAN 20:     192.168.20.0/24   (NUR der Futro: AdGuard, NPM, CrowdSec, DDNS)
IoT VLAN 30:     192.168.30.0/24   (Zigbee-Geraete via Z2M, Kameras, Smart-Home)
Gast VLAN 40:    192.168.40.0/24   (nur Internet)
WG-Netz:        10.10.0.0/24     (WireGuard-Clients)

Feste IPs:
<FUTRO_IP>  = 192.168.20.5   (DMZ! Edge-Box: AdGuard, NPM, CrowdSec, DDNS)
<SRV_IP>    = 192.168.10.10  (Proxmox-Hauptserver, Trusted)
<T630_IP>   = 192.168.10.20  (HA, MQTT, Z2M, Trusted)
```

DMZ-Firewall-Regeln auf der UDM (5 Regeln, einmalig einrichten):

#	Richtung	Regel	Zweck
1	Internet → DMZ	erlauben: TCP 80, 443 → `<FUTRO_IP>`	Exposition minimal
2	DMZ → Server-VLAN	erlauben: TCP `<NC_PORT>`, `8123` → `<SRV_IP>`/`<T630_IP>`	NPM darf NUR die Backends erreichen
3	DMZ → Management (UDM, Proxmox UI, T630 UI)	**BLOCKIEREN**	Kompromittierter Futro sieht keine Admin-UIs
4	DMZ → IoT	**BLOCKIEREN**	Kein Sprungbrett zu Kameras/Smart-Home
5	Alle → DMZ	erlauben: UDP 53 (DNS → AdGuard); Admin-Zugriff auf Futro nur aus Trusted/VPN	AdGuard-Dienst für alle Netze

UDM-Konfiguration (UI): Networks → Create VLAN Network (DMZ, VLAN-ID 20, Subnet 192.168.20.0/24, kein DHCP nötig – Futro bekommt statische IP) → Firewall → Rules nach obiger Tabelle anlegen (regelmäßig prüfen: DMZ darf initiativ NUR zu Regel-2-Zielen).

Futro anschließen: Switch-Port, an dem der Futro hängt, in der UDM auf Port-Profil DMZ/VLAN 20 setzen (Networks → Switch-Port-Konfiguration).

Architektur-Notiz (Beschluss v1.2.0): TrueNAS als NAS-Schicht wurde geprüft und **bewusst verworfen** – ZFS läuft direkt auf Proxmox, eine zusätzliche NAS-VM würde nur Verwaltungsaufwand und RAM kosten, ohne Sicherheits- oder Datenmehrwert. Daten liegen in plain ZFS-Datasets, eine spätere Migration bleibt jederzeit möglich.

0.6–0.7 UDM Pro härten (Konsole/GUI-Schritte)

1. Unifi-UI → **Settings** → **Internet Security / Telemetrie**: Datenfreigabe minimieren

2. **Settings** → **System** → **Remote Access**: Cloud-Fernverwaltung **deaktivieren**

3. Unifi-Konto: **2FA aktivieren** (account.ui.com → Security)

4. Settings → System → Firmware: Auto-Update **aus**, nur Benachrichtigung; Updates manuell im Wartungsfenster

5. Firewall-Regel: Management (UDM-UI 443/8443, SSH 22) nur aus `<NETZ>` und `<WG_NETZ>` erlauben

6. Keine Port-Forwards auf UDM-Dienste (nur spaeter 80/443 → Futro)

0.8 WireGuard auf der UDM

1. Unifi-UI → **Settings** → **VPN** → **WireGuard** → Server erstellen

2. Port: `<WG_PORT>`, Netz: `10.10.0.0/24`

3. Client (z. B. Handy) hinzufügen, Konfigurations-QR scannen

4. UDM-Firewall: `<WG_NETZ>` darf Management + Vaultwarden-Host erreichen

✔ *Abschlusstest Phase 0*

```
# Restore-Test (0.4) lieferte "RESTORE OK"
grep -c "error" /var/log/restic.log || echo "keine Fehler im Log"
```

■ Restore-Test bestanden

■ **Inventur mit Seriennummern erstellt (0.0a), SMART aller Pool-Kandidaten unauffällig (0.0b)**

■ **Container-Doku + Unraid-Flash-Backup außerhalb des Servers abgelegt (0.0c/0.0d)**

■ **Vaultwarden-Export (verschlüsselt, .json) + alle Secrets extern gesichert (0.0e)**

■ UDM von extern NICHT erreichbar (Handy im Mobilfunk: `https://<WAN-IP>` scheitert)

■ 2FA-Login auf Unifi-Konto funktioniert

■ WireGuard-Verbindung von extern aufgebaut; Management + künftiger Vaultwarden-Zugang über VPN erreichbar

Phase 1 – Futro S920 als Edge-Box

1.1 Debian installieren & Grundhärtung

```
# Debian netinst (aktuelles stable) von debian.org als USB-Stick:
#   dd if=debian-netinst.iso of=/dev/sdX bs=4M status=progress
# Installation: nur SSH-Server + Standard-System, Benutzer <USER> anlegen

# Nach Installation auf dem Futro:
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y sudo curl git

# SSH haerten: nur Key-Auth, kein root-login
sudo sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sudo sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sudo systemctl restart ssh

# Lokalen Key hinterlegen (von deinem Rechner):
#   ssh-copy-id <USER>@<FUTRO_IP>

# Automatische Sicherheitsupdates
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades

# Statische IP setzen (z. B. /etc/network/interfaces oder netplan je nach Setup):
# Beispiel systemd-networkd (/etc/systemd/network/10-lan.network)
sudo tee /etc/systemd/network/10-lan.network <<'EOF'
[Match]
Name=enp1s0
[Network]
Address=<FUTRO_IP>/24
Gateway=<GATEWAY_IP>
DNS=127.0.0.1   # AdGuard laeuft spaeter lokal
EOF
sudo systemctl restart systemd-networkd
```

1.2 Docker + Compose

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker <USER>
# neu einloggen, dann:
mkdir -p ~/edge/{adguard,npm,crowdsec,ddns}
```

1.3 AdGuard migrieren (vom T630 auf den Futro)

```
# Auf dem T630: AdGuard-Config exportieren
docker exec adguard /bin/sh -c 'tar czf - /opt/adguardhome/conf' > adguard-backup.tar.gz
scp adguard-backup.tar.gz <USER>@<FUTRO_IP>:~/edge/adguard/

# Auf dem Futro:
cd ~/edge/adguard && tar xzf adguard-backup.tar.gz
# In conf/yaml: listen-port 53 pruefen, gebundene IP auf <FUTRO_IP> setzen

docker run -d --name adguard --restart always \
  -p 53:53/tcp -p 53:53/udp \
  -v ~/edge/adguard/opt/adguardhome/conf:/opt/adguardhome/conf \
  -v ~/edge/adguard/opt/adguardhome/work:/opt/adguardhome/work \
  adguard/adguardhome:latest

# Test vom Client (DNS-Server manuell auf <FUTRO_IP> stellen oder per DHCP):
dig @<FUTRO_IP> example.com
```

1.4 Alter AdGuard auf dem T630 (erst nach Abschlusstest stoppen!)

```
# Auf dem T630 (erst nachdem alles ueber den Futro laeuft):
docker stop adguard && docker rm adguard
```

1.5 CrowdSec + NPM

```
cd ~/edge/crowdsec
cat > docker-compose.yml <<'EOF'
services:
  crowdsec:
    image: crowdsecurity/crowdsec:latest
    container_name: crowdsec
    restart: always
    environment:
      COLLECTIONS: "crowdsecurity/nginx-proxy-manager crowdsecurity/http-cve"
    volumes:
      - ./config:/etc/crowdsec
      - ./data:/var/lib/crowdsec
      - nginx-logs:/var/log/nginx:ro
  npm:
    image: jc21/nginx-proxy-manager:latest
    container_name: npm
    restart: always
    ports:
      - "80:80"
      - "443:443"
      - "127.0.0.1:81:81"
    volumes:
      - ./npm-data:/data
      - ./npm-letsencrypt:/etc/letsencrypt
volumes:
  nginx-logs:
EOF
docker compose up -d

# Bouncer registrieren (API-Key generieren):
docker exec crowdsec cscli bouncers add npm-bouncer
# -> Ausgabe: <BOUNCER_KEY>

# Middleware in NPM: Stream-Config + CrowdSec-Bouncer (Plugin) einbinden:
# Im NPM-Admin-UI (http://<FUTRO_IP>:81, admin/changeme sofort aendern!)
# unter Settings -> crowdsec die <BOUNCER_KEY> eintragen.

# CrowdSec-Status pruefen:
docker exec crowdsec cscli bouncers list
docker exec crowdsec cscli alerts list
```

1.6 DDNS-Updater

```
cd ~/edge/ddns
cat > docker-compose.yml <<'EOF'
services:
  ddns:
    image: qmcgaw/ddns-updater
    container_name: ddns
    restart: always
    ports:
      - "127.0.0.1:8000:8000"
    volumes:
      - ./data:/updater/data
EOF
# In data/config.json (Beispiel fuer DuckDNS; Provider-spezifisch anpassen):
cat > data/config.json <<'EOF'
{
  "settings": [],
  "answers": ["<DOMAIN>"],
  "providers": [{
    "provider": "duckdns",
    "zone": "<DOMAIN>",
    "domain": "<DOMAIN>",
    "token": "<DDNS_TOKEN>"
  }]
}
EOF
docker compose up -d
curl -s http://127.0.0.1:8000/update   # manuelles Update triggern, dann Provider-Konsole pruefen
```

1.7 NPM Proxy-Hosts anlegen (im Admin-UI `http://<FUTRO_IP>:81`)

• Proxy-Host `cloud.<DOMAIN>` → `http://<SRV_IP>:<PORT_NEXTCLOUD>` (Let's-Encrypt-Zertifikat, HTTP-Verifikation)

• Proxy-Host `ha.<DOMAIN>` → `http://<T630_IP>:8123`

• Erzwingen: „Websockets Support" für HA aktivieren

1.8 Cutover Port-Forwards (UDM-UI)

1. Alte Port-Forwards 80/443 (auf Unraid-NPM) **löschen**

2. Neue Port-Forwards: 80 + 443 → `<FUTRO_IP>`

3. Von extern (Mobilfunk) testen:

• `https://cloud.<DOMAIN>` → Nextcloud-Login

• `https://ha.<DOMAIN>` → HA-Login

1.9 Vaultwarden nur via VPN

1. NPM: Proxy-Host `vault.<DOMAIN>` → `http://<SRV_IP>:<PORT_VAULTWARDEN>`

2. NPM: Access List „vpn-only" anlegen: erlaubt `10.10.0.0/24` (WG-Netz), alle anderen blockiert

3. Access List dem Proxy-Host zuweisen

```
# Test von extern (ohne VPN) – MUSS scheittern:
curl -k --max-time 5 https://vault.<DOMAIN> && echo "FEHLER: erreichbar!" || echo "OK: blockiert"
# Test im VPN – muss funktionieren:
curl -k https://vault.<DOMAIN> | head -3
```

1.10 Futro-Image fuer Kaltreserve + Config-Backup

```
# Compose-Configs ins restic-Backup:
source /etc/restic-env
restic backup ~/edge --tag futro-config

# SSD-Image mit Clonezilla (von USB-Stick booten, Futro-SSD -> externen USB-Stick klonen)
# Alternativ live (Futro im Single-User/Recovery), grob:
#   sudo dd if=/dev/sda of=/media/usb/futro-image.img bs=4M status=progress
```

✔ *Abschlusstest Phase 1*

- ■ `dig @<FUTRO_IP> example.com` liefert Antworten; Internet auf allen Geraeten (Trusted/IoT/Gast) funktioniert
- ■ T630-AdGuard gestoppt, DNS laeuft stabil ueber Futro
- ■ `https://cloud.<DOMAIN>` und `https://ha.<DOMAIN>` extern erreichbar, gueltiges Zertifikat
- ■ CrowdSec: `cscli decisions add --ip 1.2.3.4` → Test-Zugriff von dort (oder mit curl von extern gegen NPM mit gebannter Test-IP) wird geblockt; danach `cscli decisions remove --ip 1.2.3.4`
- ■ Vaultwarden ohne VPN blockiert, mit VPN erreichbar (curl-Tests oben)
- ■ DDNS-Eintrag beim Provider stimmt mit aktueller WAN-IP ueberein
- ■ Clonezilla-/Image-Backup liegt vor, Kaltreserve dokumentiert

Phase 2 – Hauptserver: Unraid → Proxmox

2.1 Finale Vollbackups (VOR dem Umbau, Unraid laeuft noch)

```
# Auf Unraid: Daten-Sync zur Hetzner-Box (restic, siehe Phase 0) einmal manuell:
/usr/local/bin/backup-offsite.sh

# ZUSAETZLICH auf eine externe Platte als komplette zweite Kopie (3-2-1!):
# ACHTUNG: Formatierung loescht alles auf dem Datentraeger,
# vorher zwingend mit lsblk identifizieren:
lsblk
parted /dev/<EXT_DEV> --script mklabel gpt mkpart primary ext4 0% 100%
mkfs.ext4 -L migration-backup /dev/<EXT_DEV>1
mkdir -p /mnt/disks/extern
mount /dev/<EXT_DEV>1 /mnt/disks/extern

# Nextcloud aufraeumen (spart Platz, reduziert Kopiermenge):
docker exec -u www-data <NEXTCLOUD_CONTAINER> php occ trashbin:cleanup --all-users
docker exec -u www-data <NEXTCLOUD_CONTAINER> php occ versions:cleanup
docker exec -u www-data <NEXTCLOUD_CONTAINER> php occ files:scan --all
docker image prune -a

# DB-Dumps der datenbankgestuetzten Dienste ziehen:
docker exec <DB_CONTAINER> mysqldump -u <DB_USER> -p<DB_PW> <DB_NAME> \
  > /mnt/user/<SHARE>/db-dumps/nextcloud-db.sql
# (analog fuer Paperless / Traccar / Immich)

# Vollstaendiger rsync (-aHAX: Hardlinks, ACLs, xattrs erhalten):
rsync -aHAX --info=progress2 --log-file=/mnt/disks/extern/rsync-backup.log \
  /mnt/user/ /mnt/disks/extern/backup/
# Bei Platzmangel optional: --exclude='<ERSETZBARER_SHARE>/'

### VERIFIKATION (Pflicht - Backup zaehlt erst mit leerer Differenz!):
rsync -aHAXcn --itemize-changes /mnt/user/ /mnt/disks/extern/backup/
# -> Ausgabe MUSS leer sein

# SHA-256-Stichproben vergleichen (3-5 groessere Dateien):
sha256sum /mnt/user/nextcloud/<STICHPROBE_DATEI>
sha256sum /mnt/disks/extern/backup/nextcloud/<STICHPROBE_DATEI>

# Inventar fuer spaetere Gegenpruefung:
cd /mnt/disks/extern/backup
find . -type f | wc -l > ../dateianzahl.txt
du -sh . > ../groesse.txt
cd ..
sync
umount /mnt/disks/extern
# Platte beschriften und physisch verwahren (2. Kopie, unabhaengig von Hetzner!)
```

2.1a Alt-Platten bleiben EINGEBAUT (Rollback-Sicherung, Beschluss v1.2.0)

Die Unraid-Array-Platten bleiben **eingebaut** und unangetastet. Proxmox wird auf der

250-GB-SSD installiert; die ungenutzte 3-TB-HDD wird als **temporärer Migrationspool

`bulk`** verwendet. Die Array-Platten werden erst nach dem 14-Tage-Gate (siehe 2.9a)

für den `tank`-Mirror freigegeben.

```
# 1. Unraid sauber herunterfahren: Main > Array > Stop, dann Tools > Shutdown
# 2. Platten beschriften (UNRAID-DISK1, UNRAID-DISK2, UNRAID-CACHE
#    + Seriennummer notieren) - sie bleiben EINGEBAUT
# 3. Unraid-USB-Stick separat aufbewahren, NICHT ueberschreiben
# 4. 250-GB-SSD und die ungenutzte 3-TB-HDD einbauen, NVMe bleibt drin
# 5. Server startet; BIOS/UEFI zeigt alle Datentraeger -> weiter mit 2.2
```

NOTFALL-ROLLBACK (Array-Platten wurden nicht angefasst, Unraid-Stick vorhanden):

```
# 1. Proxmox-Host sauber herunterfahren
# 2. Boot-Reihenfolge im BIOS auf Unraid-USB-Stick umstellen (oder SSD abziehen)
# 3. Unraid starten, Array starten -> Container laufen mit Stand aus 2.1
# 4. temporaeren bulk-Pool unangetastet lassen (er haelt die Migrationskopie)
```

Wichtig fuer Phase 3 (tank-Mirror): Vor dem Ueberbuehren der beiden Array-Platten

muessen die Daten verifiziert auf `bulk` UND extern (USB-Platte + Hetzner) liegen und

das 14-Tage-Gate bestanden sein - erst dann werden die Array-Platten geloescht und

als `tank`-Mirror neu angelegt. Eine Platte nie gleichzeitig als Zwischenpool

und Backup-Pool verwenden.

2.2 Proxmox installieren

1. Proxmox-VE-ISO von proxmox.com laden, USB-Stick: `dd if=proxmox.iso of=/dev/sdX bs=4M`

2. Boot: 250-GB-SSD als Ziel, ext4 (Boot-Platte), Land: de, Keyboard: de

3. Management-IP: `<SRV_IP>/24`, Gateway `<GATEWAY_IP>`

4. Nach Installation: Web-UI `https://<SRV_IP>:8006`

5. Repositories umstellen (kein Enterprise-Repo ohne Abo):

```
# Auf Proxmox-Host:
sed -i.bak 's/^deb/#deb/' /etc/apt/sources.list.d/pve-enterprise.list
echo "deb http://download.proxmox.com/debian/pve bookworm pve-no-subscription" \
  > /etc/apt/sources.list.d/pve-no-subscription.list
apt update && apt full-upgrade -y
```

2.3 ZFS-Storage anlegen (nach Installation, in Proxmox-Shell)

```
# 1TB NVMe = 'fast' (LXC-Disks, DBs)
zpool create -o ashift=12 fast /dev/disk/by-id/nvme-<NVME_ID>

# 3TB HDD = 'bulk' (Files, Docs, vzdump)
zpool create -o ashift=12 bulk /dev/disk/by-id/ata-<HDD_ID>

# Datasets (saubere Snapshot-/Quota-Politik je Dataset):
zfs create bulk/nextcloud
zfs create bulk/paperless
zfs create bulk/immich
zfs create bulk/cameras      # Snapshot-Politik: deaktiviert/24h (s. 2.4)
zfs create bulk/vzdump

# Proxmox-Storage einbinden (GUI: Datacenter -> Storage -> Add):
#   fast  : ZFS-Storage (oder Verzeichnis /fast via Verwaltung)
#   bulk/vzdump : "Backup"-Typ
```

2.4 zfs-auto-snapshot

```
apt install -y zfs-auto-snapshot
# Standard: alles snapshots. Kameras begrenzen:
zfs set com.sun:auto-snapshot=false bulk/cameras
# (oder frequelle Politiken je Dataset anpassen: zfs set com.sun:auto-snapshot:daily=true ...)
```

2.5 LXC-Migration – Reihenfolge & generisches Muster

Je Dienst (Vaultwarden → Paperless → Traccar → Immich → Nextcloud):

```
# 1. Template herunterladen (GUI oder CLI):
pveam update && pveam available --section system
pveam download local debian-12-standard_12.7-1_amd64.tar.zst

# 2. LXC anlegen (GUI: Create CT, unprivileged, nesting fuer Docker-frei LXCs)
#    oder CLI-Beispiel Vaultwarden:
pct create 101 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
  --hostname vaultwarden --memory 512 --swap 512 \
  --net0 name=eth0,bridge=vmbr0,ip=<VAULT_IP>/24,gw=<GATEWAY_IP> \
  --rootfs fast:8 --unprivileged 1 --features nesting=1 \
  --start 1

# 3. Im LXC: Dienst installieren (Vaultwarden-Beispiel, Docker im LXC):
pct enter 101
apt update && apt install -y curl
curl -fsSL https://get.docker.com | sh
mkdir -p /opt/vw && cd /opt/vw
cat > docker-compose.yml <<'EOF'
services:
  vaultwarden:
    image: vaultwarden/server:latest
    restart: always
    environment:
      DOMAIN: "https://vault.<DOMAIN>"
      SIGNUPS_ALLOWED: "false"      # Geschlossen halten!
    volumes:
      - ./data:/data
    ports:
      - "<PORT_VAULTWARDEN>:80"
EOF
docker compose up -d

# 4. Daten migrieren (von Unraid auf LXC):
rsync -avh --progress <USER>@<UNRAID_IP>:/mnt/user/vaultwarden/ /opt/vw/data/
# bzw. DB-Dumps bei DB-gestuetzten Diensten:
#   docker exec <DB_CONTAINER> mysqldump ... > dump.sql  (Nextcloud!)

# 5. Intern testen: http://<VAULT_IP>:<PORT_VAULTWARDEN> (Login, Sync-Test)

# 6. NPM-Proxy auf neue IP umstellen (NPM-UI), dann alten Unraid-Dienst stoppen.
```

Nextcloud-Spezifisch (Downtime-Fenster):

```
# Vorab (im Betrieb) - Vorab-Sync der Files:
rsync -avh --progress <USER>@<UNRAID_IP>:/mnt/user/nextcloud/ /bulk/nextcloud/

# Im Wartungsfenster:
# Unraid: Nextcloud in Wartungsmodus:
sudo -u www-data php occ maintenance:mode --on
# Finaler Delta-Sync:
rsync -avh --delete --progress <USER>@<UNRAID_IP>:/mnt/user/nextcloud/ /bulk/nextcloud/
# DB-Dump auf Unraid ziehen und im neuen LXC importieren:
docker exec <DB_CONTAINER> mysqldump -u <DB_USER> -p<DB_PW> nextcloud > nextcloud-db.sql
# Neuer LXC: DB-Container starten, Dump einspielen:
docker exec -i <DB_CONTAINER> mysql -u <DB_USER> -p<DB_PW> nextcloud < nextcloud-db.sql
# Config anpassen (DB-Host, Datenpfad), maintenance:mode --off, dann testen.
```

Immich (nach Nextcloud-Vorbereitung, gleiche Phase):

```
# LXC mit mehr Ressourcen (ML braucht CPU): 4 vCPU, 4-8GB RAM
# Files -> bulk/immich, DB/Thumbs -> fast (NVMe)
# Docker-Compose aus offiziellem Immich-Repo, .env mit Pfaden auf die Datasets.
```

2.6 Reolink-Aufnahmen auf Netzwerk-Target

```
# SMB-Share bereitstellen (Samba auf Proxmox-Host oder eigener kleiner LXC):
zfs create bulk/cameras
apt install -y samba
# /etc/samba/smb.conf Eintrag:
cat >> /etc/samba/smb.conf <<'EOF'
[cameras]
   path = /bulk/cameras
   valid users = <CAM_USER>
   read only = no
EOF
smbpasswd -a <CAM_USER>
systemctl restart smbd

# Reolink-Kamera/Web-UI: Aufnahmen -> FTP/SMB-Upload auf \\<SRV_IP>\cameras
# In Reolink-App (bzw. Home Assistant Reolink-Integration) pruefen, dass Clips ankommen.
```

2.7 T630-Backups anbinden

```
# Auf Proxmox-Hauptserver: NFS-Server oder simpler SSH-Zielbereich:
mkdir -p /bulk/vzdump/t630
# Auf dem T630 (Proxmox): Backup-Job einrichten:
#   /etc/pve/jobs.cfg oder GUI: Datacenter -> Backup -> Add
#   Storage: NFS-Mount auf <SRV_IP>:/bulk/vzdump/t630 (oder SSH/SFTP-Ziel)
# Test-Backup eines LXC auf dem T630:
vzdump 100 --storage <NFS_STORAGE> --mode snapshot
```

2.8 Regelmäßige Proxmox-Backups (Hauptserver)

```
# GUI: Datacenter -> Backup -> Add:
#   Storage: bulk/vzdump, Schedule: So 03:00, Mode: snapshot,Retention: 4 Wochen
# Alternativ CLI-Test eines LXCs:
vzdump 101 --storage <BULK_STORAGE> --mode snapshot --compress zstd
```

2.9a Beobachtungszeitraum vor Freigabe der Alt-Platten (14 Tage)

Mindestens **14 Tage** nach dem Cutover beobachten, bevor die alten Unraid-Platten

freigegeben werden (siehe 3.4):

```
zpool status                 # beide Pools ONLINE, keine Fehler
zpool list -o health         # Scrub-Ergebnisse kontrollieren
smartctl -H /dev/<POOL_HDD>  # Plattenzustand
grep -i error /var/log/restic.log | tail   # Hetzner-Backup fehlerfrei?
# Alle externen Dienste erreichbar? (Handy-Check / Uptime-Kuma)
```

✔ Abschlusstest Phase 2

```
zfs list -t snapshot | head -20      # Snapshots werden automatisch erzeugt
vzdump <CTID> --storage <BULK_STORAGE> --mode snapshot  # manueller Test-Backup
pct restore 999 <BACKUP-DATEI> --storage fast   # Restore-Test in eine Test-CT (danach loeschen!)
```

■ Vaultwarden, Paperless, Traccar, Immich, Nextcloud: alle laufen auf Proxmox, Login/Upload/Sync getestet

■ Vaultwarden weiterhin VPN-only (curl-Test wiederholen)

■ Reolink-Clips landen auf `bulk/cameras`

■ vzdump-Restore in Test-CT erfolgreich

■ **rsync-Verifikation des externen Backups leer (keine Differenzen), SHA-256-Stichproben identisch (2.1)**

■ **Alte Unraid-Platten ausgebaut, beschriftet, antistatisch gelagert; Notfall-Rollback dokumentiert (2.1a)**

■ T630-Backup-Job laeuft, Dateien liegen auf dem Hauptserver

■ Unraid stillgelegt (keine laufenden Dienste, Port-Forwards restlos entfernt)

Phase 3 – Finalisierung

3.1 tank-Mirror aus den freigegebenen Array-Platten + backup-Pool (Beschluss v1.2.0)

Zeitliche Abfolge STRIKT einhalten:

1. 14-Tage-Gate bestanden (2.9a), Daten verifiziert auf `bulk` + extern + Hetzner

2. Zwei alte Array-Platten freigeben -> `tank` als ZFS-Mirror anlegen

3. Daten von `bulk` nach `tank` migrieren und verifizieren (rsync + Scrub)

4. ERST DANACH `bulk` leeren und als dedizierten `backup`-Pool neu anlegen

```
# Vorher: Seriennummern der beiden Array-Platten erneut abgleichen!
ls -l /dev/disk/by-id/ | grep -E "<ARRAY_HDD1_ID>|<ARRAY_HDD2_ID>"

# tank-Mirror anlegen (Achtung: ueberschreibt die beiden alten Array-Platten!):
zpool create -o ashift=12 tank mirror /dev/disk/by-id/<ARRAY_HDD1_ID> /dev/disk/by-id/<ARRAY_HDD2_ID>
zfs set compression=lz4 tank
zfs set atime=off tank
zfs create tank/nextcloud tank/paperless tank/immich tank/documents tank/cameras
zpool status -v tank

# Datenmigration bulk -> tank (Dienste vorher stoppen!):
# Erst Dry-Run, dann Echtlauf, dann Verifikation:
rsync -aHAXn --info=progress2 --itemize-changes /bulk/nextcloud/ /tank/nextcloud/
rsync -aHAX --info=progress2 /bulk/nextcloud/ /tank/nextcloud/
rsync -aHAXcn --itemize-changes /bulk/nextcloud/ /tank/nextcloud/   # MUSS leer sein
# (analog fuer alle weiteren Datasets; danach Scrub:)
zpool scrub tank

# Erst wenn ALLES auf tank verifiziert liegt UND Dienste auf tank laufen:
zpool destroy bulk
zpool create -o ashift=12 backup /dev/disk/by-id/<TEMP_HDD_ID>
zfs set compression=lz4 backup
zfs set atime=off backup
zfs create backup/tank-repl
zfs create backup/fast-repl
zpool status -v backup
```

3.1a sanoid/syncoid – nächtliche ZFS-Replikation (Beschluss v1.2.0)

```
# Installation (Debian/Proxmox-Paketquellen):
apt install -y sanoid syncoid

# /etc/sanoid/sanoid.conf anlegen:
cat > /etc/sanoid/sanoid.conf <<'EOF'
[tank/nextcloud]
use_template = production

[tank/paperless]
use_template = production

[tank/immich]
use_template = production

[tank/documents]
use_template = production

[tank/cameras]
use_template = camera

[fast]
use_template = production

[template_production]
frequently = 0
hourly = 24
daily = 14
monthly = 6

[template_camera]
frequently = 0
hourly = 0
daily = 1
monthly = 0
EOF

# Snapshots sofort testen:
sanoid --take-snapshots
zfs list -t snapshot | head

# Replikation manuell testen (vor dem Cron):
syncoid --recursive tank backup/tank-repl
syncoid fast backup/fast-repl

# Nachtcron einrichten:
echo "15 2 * * * root /usr/sbin/sanoid --take-snapshots && /usr/sbin/syncoid --recursive tank backup/tank-repl && /us
  > /etc/cron.d/syncoid-nightly
chmod 644 /etc/cron.d/syncoid-nightly

# WICHTIG (Ransomware-Schutz): backup-Pool danach read-only setzen:
zfs set readonly=on backup/tank-repl
zfs set readonly=on backup/fast-repl
# syncoid hebt readonly beim Schreiben kurz auf; vor Manipulation schuetzt es trotzdem.

# Restore-Test aus dem Replikat (PFLICHT, nicht nur Status ablesen):
zfs send backup/tank-repl/nextcloud@autosnap_$(date +%F)* \
  | zfs receive tank/restore-test
ls /tank/restore-test && zfs destroy -r tank/restore-test
```

3.1b Externe USB-Platte als lokales Backup-Tier (Beschluss v1.2.0)

Die vorhandene 1-TB-USB-3.0-Platte wird **dauerhaft angeschlossen** und als

zweites lokales restic-Ziel genutzt (schnelle Wiederherstellung ohne Hetzner).

Hinweis: Dauerhaft verbunden = KEINE Offline-Kopie; als Offline-Disziplin gilt

der restic-Historie auf Hetzner plus optionaler monatlicher manueller Pull.

```
# Platte formatieren (ACHTUNG: loescht alles, vorher lsblk!):
lsblk
mkfs.ext4 -L usb-backup /dev/<USB_DISK>1
mkdir -p /mnt/usb-backup

# /etc/fstab Eintrag (dauerhaft, mit nofail fuer Robustheit):
echo "/dev/disk/by-uuid/<USB_UUID> /mnt/usb-backup ext4 defaults,nofail,x-systemd.device-timeout=30 0 2" >> /etc/fsta
mount -a

# Zweites restic-Repository initialisieren:
export RESTIC_REPOSITORY="/mnt/usb-backup/restic"
export RESTIC_PASSWORD="<RESTIC_PASSPHRASE>"   # gleiche Passphrase wie Hetzner
restic init

# Backup-Skript erweitern (nach dem Hetzner-Run):
cat >> /usr/local/bin/backup-offsite.sh <<'EOF'
# Lokale USB-Kopie (schnelle Wiederherstellung ohne Cloud):
RESTIC_REPOSITORY=/mnt/usb-backup/restic restic backup /tank/nextcloud /tank/paperless \
  /tank/immich /tank/documents /fast/vmdata --tag usb-local
RESTIC_REPOSITORY=/mnt/usb-backup/restic restic forget --keep-daily 7 --prune
EOF

# OPTIONAL (empfohlen, echte Offline-Kopie): monatlich 10 Min. abziehen
# - Platte abstecken -> separate volle Kopie nicht noetig, Restic-Historie reicht
# - Kalender-Reminder einrichten: "1. des Monats: USB-Platte kurz pruefen + abziehen"
```

3.2 Kamera-Dataset aufräumen

```
# Alte Aufnahmen automatisch entfernen (Platz!):
cat > /etc/cron.daily/camcleanup <<'EOF'
#!/bin/bash
find /bulk/cameras -type f -mtime +14 -delete
EOF
chmod +x /etc/cron.daily/camcleanup
```

3.3 Monitoring/Alarmierung

```
# Healthchecks.io (kostenlos): Check anlegen, URL = <HEALTHCHECK_URL>
# Am Ende des Backup-Skripts:
echo 'curl -fsS --retry 3 <HEALTHCHECK_URL> > /dev/null' >> /usr/local/bin/backup-offsite.sh
# -> Bleibt ein Ping aus, kommt eine E-Mail. Damit schlaeft kein silently totes Backup.
```

3.4 Alte Platten erst nach ALLEN Pruefungen freigeben

```
# Erst nach 14-taegiger Beobachtung (2.9a) + bestandenem Restore-Drill + nmap-Test:
# - Platten sicher loeschen: blkdiscard (NVMe) bzw. dd/shred (HDD)
# - ODER (Empfehlung) dritte 3-TB-HDD dem Pool zufuehren (siehe 3.1)
# - Externe Backup-Platte als Offline-Kopie weiterverwenden
# - Unraid-USB-Stick archivieren (nicht loeschen - kostet nichts)
```

3.5 Externer Port-Scan als Abschlusscheck

```
# Von extern (z. B. Handy im Mobilfunk oder VPS):
nmap -Pn -p- <DEINE_WAN_IP>
# Erwartung: nur 443 (und ggf. WireGuard-Port) offen, sonst NICHTS

# Auf dem Hauptserver: offene Dienste pruefen:
ss -tulpn
```

3.6 Restore-Drill (der wichtigste Schritt!)

```
source /etc/restic-env
restic snapshots
# Nextcloud + Vaultwarden + T630-LXC aus Hetzner-Backup in Test-Umgebung:
restic restore latest --target /tmp/drill --path /bulk/nextcloud
restic restore latest --target /tmp/drill --path /opt/vw/data
# Funktionalitaet pruefen: DB-Import in Test-DB, Vaultwarden-Items lesbar?
rm -rf /tmp/drill
```

✔ *Abschlusstest Phase 3 (Gesamtabnahme)*

■ Mirror + Backup-HDD eingebunden, vzdump schreibt auf dediziertes Ziel (getrennt vom Datenpool)

■ Restore-Drill erfolgreich (Nextcloud, Vaultwarden, T630-LXC funktional geprueft)

■ **14-Tage-Beobachtung fehlerfrei (zpool status, SMART, restic-Log, Dienste) – erst DANN Alt-Platten freigeben (3.4)**

■ **nmap von extern zeigt nur 443 (+ ggf. WireGuard), keine weiteren offenen Ports (3.5)**

■ Healthchecks-Ping kommt an (Test-Alarm einmal provoziert)

■ Kamera-Cleanup-Job vorhanden (`ls /etc/cron.daily/camcleanup`)

■ Doku: IP-Plan, Restore-Anleitung, USB-Stick-Verwahrungsorte in Paperless abgelegt

■ Kaltreserve-Futro: Image + Wiederinbetriebnahme-Anleitung (<30 Min) dokumentiert

Backup-Übersicht (3-2-1)

Stufe	Mechanismus	Ziel	Rhythmus
1	ZFS-Snapshots (zfs-auto-snapshot)	lokal (NVMe + HDD)	stündlich/täglich
2	vzdump aller LXCs (beider Hosts)	backup-HDD im Hauptserver	wöchentlich (So 03:00)
3	restic verschlüsselt (Key auf USB-Stick)	Hetzner Storage Box	täglich (02:00)